

Documentación Proyecto Integrador

1. 📌 Descripción general del proyecto

El presente proyecto tiene como objetivo diseñar e implementar un **pipeline de datos completo en AWS** utilizando datasets reales de transporte de la ciudad de Nueva York (TLC – Taxi & Limousine Commission).

El pipeline cubre las siguientes etapas:

- Ingesta de datos desde fuente pública
- Procesamiento ETL (limpieza y transformación)
- Procesamiento distribuido con Apache Spark
- Generación de capa analítica
- Visualización interactiva mediante Streamlit

Los datos utilizados provienen del portal oficial de NYC:

👉 <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

Los datasets incluyen:

- Yellow Taxi
- Green Taxi
- FHV (For-Hire Vehicles)
- FHVHV (High Volume)

2. 📁 Arquitectura general

El pipeline se diseñó bajo una arquitectura modular basada en AWS:

Componentes principales:

- **Amazon S3**
 - `raw_data`: datos originales en formato Parquet

- `clean_data` : datos limpios
- `analytics` : datos agregados para visualización
- **AWS Lambda**
 - Extracción automatizada de datasets
- **EC2**
 - Ejecución de scripts ETL y Spark
- **Apache Spark + Hadoop**
 - Procesamiento distribuido de grandes volúmenes
- **Streamlit (Dockerizado)**
 - Visualización interactiva

3. Ingesta de datos

Fuente

Los datos se obtuvieron del portal oficial de NYC TLC en formato **Parquet**, optimizado para análisis.

Proceso de extracción

Se implementó una **función Lambda en AWS** que:

1. Consulta las URLs públicas de los datasets
2. Descarga los archivos `.parquet`
3. Los almacena en:

```
s3://bucket/raw_data/tlc/{dataset}/year=YYYY/month=MM/
```

Ventajas del enfoque

- Automatización completa
 - Datos organizados por partición
 - Formato eficiente (Parquet)
-

4. 🛠️ Procesamiento ETL (Limpieza y Transformación)

Se diseñaron dos enfoques:

4.1 Script Python (datasets pequeños)

Se utilizó para:

- `green`
- `fhv`
- `yellow` (hasta cierto punto)

Funcionalidades:

- Eliminación de duplicados
 - Normalización de columnas
 - Conversión de fechas
 - Cálculo de duración de viaje
 - Limpieza de valores nulos
-

4.2 Script Python + Spark (datasets grandes)

Se utilizó para:

- `fhvhv`
 - `yellow` (volumen alto)
-

5. ⚙️ Procesamiento distribuido con Spark

Configuración

Se implementó Spark en EC2 utilizando:

- PySpark
 - Hadoop S3A connector
-

Características del procesamiento

- Lectura directa desde S3 (`s3a://`)
 - Procesamiento distribuido
 - Manejo de grandes volúmenes de datos
-

Transformaciones realizadas

- Estandarización de columnas
 - Conversión de timestamps
 - Generación de variables derivadas:
 - `pickup_date`
 - `pickup_hour`
 - `pickup_day_of_week`
 - Cálculo de:
 - `trip_duration_min`
-

Salida

```
s3://bucket/clean_data/tlc/{dataset}/year=YYYY/month=MM/
```

6. Capa de Analytics

Se diseñó una capa adicional optimizada para visualización:

KPIs calculados

- Total de viajes
 - Duración promedio
 - Ingreso total
 - Distancia promedio
 - Propinas promedio
 - Distribución por hora
 - Distribución por día
-

Script de analytics

Se creó un script que:

1. Lee datos desde `clean_data`
2. Aplica agregaciones
3. Genera datasets resumidos

Salida:

```
s3://bucket/analytics/tlc/{dataset}/year=YYYY/month=MM/
```

7. Visualización con Streamlit

Características principales

Se desarrolló una aplicación interactiva que:

- Permite seleccionar:
 - Tipo de dataset (`yellow` , `green` , `fhv` , `fhvhv`)
 - Año
 - Mes

Interfaz

Incluye:

- Dropdowns dinámicos
- KPIs en tiempo real
- Gráficas interactivas

KPIs mostrados

- Total de viajes
- Duración promedio
- Ingresos
- Distancia promedio

Gráficas

- Distribución por hora
 - Distribución por día
 - Tendencias temporales
-

Flujo de datos

Usuario selecciona → Streamlit → consulta analytics en S3 → muestra KPIs

Trigger Action

El comportamiento es dinámico:

- Cada cambio en el dropdown dispara una nueva consulta
 - Se actualizan automáticamente los datos y gráficos
-

8. 🐳 Dockerización de Streamlit

La aplicación fue contenerizada para facilitar despliegue.

Beneficios:

- Portabilidad
 - Reproducibilidad
 - Fácil despliegue en EC2 o cloud
-

9. 🧠 Insights esperados

El sistema permite descubrir:

- Horas pico de transporte
- Días con mayor demanda
- Relación entre distancia y costo
- Comportamiento de propinas
- Comparación entre tipos de transporte

10. Seguridad

Se implementaron buenas prácticas:

- Uso de IAM Roles
 - Acceso controlado a S3
 - Uso de credenciales por instancia
-

11. Conclusión

El proyecto demuestra:

- Implementación de un pipeline completo
- Uso de herramientas modernas (AWS, Spark, Docker)
- Procesamiento de datos reales a gran escala
- Visualización interactiva profesional